

LambdaLens

王亦凡, 任睿欣, 杨倚
数据科学与信息工程学院

April 28, 2026

Contents

1	摘要	3
2	项目背景与意义	3
3	系统总体设计	3
3.1	系统架构图	3
3.2	模块说明	4
3.3	后端	4
3.3.1	Parser 模块	4
3.3.1.1	功能	4
3.3.1.2	内部实现	4
3.3.2	AST 模块	4
3.3.2.1	功能	4
3.3.3	Evaluator 模块	4
3.3.3.1	功能	4
3.3.3.2	内部逻辑	4
3.3.4	API 模块	4
3.3.4.1	功能	4
3.3.4.2	设计理由	5
3.4	前端	5
3.4.1	功能	5
3.4.2	内部实现	5
3.4.3	设计理由	6
4	核心技术实现	6
4.1	λ 演算语法	6
4.2	AST 定义	6
4.3	β -规约算法	7
4.4	HM 类型推导 (Algorithm W)	7
4.4.1	简述	7
4.4.2	类型系统定义:	7
4.4.3	Substitutable 类型类	8
4.4.4	替换 (Substitution) 与合一 (Unification)	8
4.4.5	实例化与泛化	8
4.4.5.1	实例化 (Instantiation)	8
4.4.6	泛化 (Generalization)	8
4.4.7	模块价值	8
5	数据结构设计	9
5.1	Type: 静态类型	9

5.2	Scheme: 多态类型 (	9
5.3	Subst: 替换表	9
6	创新点	9
6.1	可视化 λ 演算规约	9
6.2	支持部分现代函数式编程语言特性的 λ 演算	9
6.3	HM 类型系统的实现	10
7	使用指南	10
7.1	快速开始	10
7.1.1	输入表达式	10
7.1.2	执行操作	10
7.1.3	使用实例库	10
7.1.4	语法帮助	10
8	安装与部署	11
8.1	环境要求	11
8.2	后端部署	11
8.3	前端部署	11
8.4	配置说明	11
9	示例演示	11
9.1	示例 1: 恒等函数	11
9.2	示例 2: 函数应用	11
9.3	示例 3: 条件表达式	12
9.4	示例 4: 递归函数	12
10	测试与验证	12
10.1	测试范围	12
11	局限性与未来工作	12
11.1	当前局限性	12
11.2	未来工作	12
12	参考文献	12

1 摘要

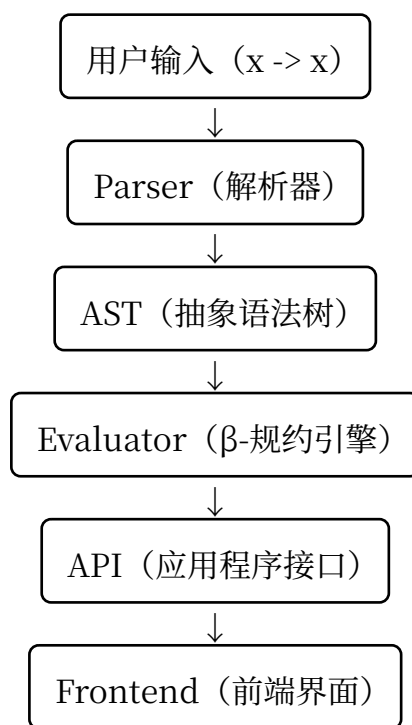
LambdaLens 是一个用于可视化 λ 演算, 逐步进行 β -规约过程的教学平台。项目采用了 HM 类型系统, 支持多种数据类型和函数定义, 且有良好的前端页面, 适合用于 λ 演算的教学与函数式编程的基本演示。

2 项目背景与意义

在编程语言的发展历程中, 函数式语言占据了重要的地位, 而 λ 演算作为函数式编程的理论基础[[1]], 对于理解函数式编程的核心概念至关重要。然而, λ 演算的抽象性常常使得初学者感到疑惑, 尤其是对于现代基于含类型 λ 演算的编程语言来说。因此, 开发一个能够可视化 λ 演算过程的教学平台, 可以有效帮助老师开展教学, 帮助学生更好的理解 λ 演算的核心概念和规约过程。不仅如此, 现代函数式编程语言的安装也是一些计算机基础薄弱的同学都难题, 开发一个在线的 Lambda 演算平台, 也能够帮助他们在不接触现代函数式编程语言繁重的工具链的情况下, 高效的学习函数式编程。并且, 国内互联网上对于 Lambda 演算的资料较少, 此项目也解决了现代 λ 演算教学资源匮乏的问题, 具有重要的教育意义和实用价值。

3 系统总体设计

3.1 系统架构图



3.2 模块说明

本系统采用分层模块化设计, 将 λ 演算表达式的解析、抽象语法树构建、 β -规约执行、可视化渲染与前端交互解耦, 形成清晰的技术结构。

3.3 后端

3.3.1 Parser 模块

3.3.1.1 功能

负责将用户输入的 λ 演算表达式解析成抽象语法树 (AST)。支持基本的 λ 演算语法, 包括变量、函数抽象和函数应用。

3.3.1.2 内部实现

- 采用递归下降解析
- 支持变量、函数抽象、函数应用
- 处理括号优先级

3.3.2 AST 模块

3.3.2.1 功能

定义 λ 表达式的内部表示结构, 作为系统的核心中间层。

3.3.3 Evaluator 模块

3.3.3.1 功能

实现 β -规约引擎, 负责执行 λ 演算表达式的规约过程, 使用应用序列策略进行求值。接受 AST, 输出规约后的 AST。

3.3.3.2 内部逻辑

- 支持应用序 (Applicative Order) 规约策略
- 实现 α -重命名避免变量捕获
- 实现替换 (Substitution) 算法

3.3.4 API 模块

3.3.4.1 功能

为前端提供统一的 HTTP 接口, 屏蔽内部实现细节。

提供接口包括:

- `/api/trace`: 返回 λ 演算表达式的规约过程的详细步骤。
 - `/api/typecheck`: 提供类型检查服务, 验证 λ 表达式的类型正确性。
 - `/api/eval`: 提供表达式求值服务, 执行 λ 演算表达式的规约。
- 这三个 API 的请求题均是

```
{ "expr": String }
```

而他们的响应体如下:

- `/api/trace`:

```
{ "steps": [...], "type": String? }
```

```
· /api/typecheck:  
  { "type": String }  
· /api/eval:  
  { "value": String, "type": String }
```

3.3.4.2 设计理由

实现前后端分离, 便于维护和扩展, 同时也为未来可能的移动端或其他平台提供接口支持。

3.4 前端

3.4.1 功能

前端模块为用户提供图形化交互界面, 负责接收用户输入的 λ 表达式, 调用后端 API 获取规约步骤、求值结果或类型信息, 并以可视化列表、卡片等形式呈现。主要功能包括:

- 表达式输入与操作选择: 支持 λ 演算语法 ($\lambda x \rightarrow expr$), 提供“单步规约”、“直接求值”、“类型推导”三个独立按钮。
- 规约过程可视化: 调用 `/api/trace`, 以步骤列表展示每一步的表达式、类型及所用规约规则 (如 β -reduction), 用户可点击任意步骤高亮查看详情。
- 直接求值: 调用 `/api/eval`, 显示最终结果及类型。
- 类型推导: 调用 `/api/typecheck`, 显示表达式的 Hindley-Milner 类型。
- 实例库与自定义规则: 左侧栏内置常用 λ 表达式实例, 并支持用户添加、编辑、删除自己的规则 (如 `add1 =  $\lambda x \rightarrow x + 1$`), 规则名称不可与保留符号冲突。自定义规则会持久化保存在浏览器本地存储中。
- 规则自动展开: 在发送请求前, 将输入表达式中的自定义规则名替换为对应的 λ 表达式 (例如 `add1 2  $\rightarrow$  ( $\lambda x \rightarrow x + 1$ ) 2`)。
- 语法帮助: 提供“语法”按钮, 点击可查看 λ 演算的基本语法说明。
- 交互优化: 左侧栏宽度可拖拽调节, 表达式修改不影响已显示结果, 错误信息有明确提示。

3.4.2 内部实现

- **技术栈:** React 18 + TypeScript + Vite, 采用函数式组件与 Hooks 管理状态。
- **组件划分:**
 - App: 主容器, 管理整体布局 (左侧实例栏 + 右侧主工作区)。
 - GrammarModal: 语法帮助弹窗组件。
 - 左侧实例栏: 展示预设规则与用户自定义规则, 支持规则的增、删、改、查操作。
 - 右侧工作区: 包含表达式输入框、三个操作按钮以及结果展示卡片。
- **状态管理:**
 - 使用 `useState` 管理当前表达式、规约结果、加载状态、错误信息。
 - 使用 `useLocalStorage` 自定义 Hook 持久化用户自定义规则列表和侧边栏宽度。
- **API 调用封装:**

- 统一 `apiCall` 函数处理 `fetch` 请求, 解析 JSON 响应, 并对错误信息进行友好提取。
- 三个功能函数 `callTrace`、`callEval`、`callTypeCheck` 分别调用对应的后端接口。
- **规则展开算法:**
 - 在调用 API 前, 对输入表达式进行正则匹配, 将规则名替换为 (规则表达式), 支持嵌套和名称长度降序排序, 避免短名误替换。
- **UI 渲染:**
 - 规约步骤列表不显示序号, 避免歧义; 每个步骤下方显示规约规则 (如有)。
 - 类型推导结果使用紫色背景 + 左边框特殊样式突出显示。
 - 所有卡片采用统一圆角、阴影, 错误信息红色背景高亮。

3.4.3 设计理由

- **前后端分离:** 前端专注用户界面与交互, 后端负责 λ 演算核心算法, 便于独立开发、测试与维护, 也为未来可能的移动端或其他平台复用 API。
- **本地持久化:** 自定义规则保存在浏览器 `localStorage` 中, 无需后端数据库, 降低部署成本, 同时保留用户个性化配置。
- **渐进增强:** 即使后端不可用, 前端仍可展示演示数据 (模拟规约步骤), 保证教学演示的基本可用性。
- **用户友好:** 拖拽调节侧边栏、表达式修改不影响已有结果等细节, 提升了教学场景下的使用流畅度。

4 核心技术实现

4.1 λ 演算语法

系统采用 Haskell 风格的 λ 演算具体语法 (Concrete Syntax) `[[2]]`, 支持变量、函数抽象与函数应用。基本形式如下:

- 变量 (Variable): `x`
- 抽象 (Abstraction): `\x -> e`
- 应用 (Application): `f x`
- 括号 (Parentheses): `f (g x)` 用于显式指定优先级

4.2 AST 定义

```
data Expr
= EInt Int
| EBool Bool
| EVar String
| ELam String Expr
| EApp Expr Expr
| ELet String Expr Expr
| ELetRec String Expr Expr
| EIf Expr Expr Expr
| EBinOp Op Expr Expr
deriving (Show, Eq)

data Op
= Add
```

```

| Sub
| Mul
| Div
| Eq
| Lt
| Gt
deriving (Show, Eq)

```

本项目并没有选择较为简单的 AST 定义, 而是选择了一个较为复杂的 AST 定义, 支持整数、布尔值、变量、函数抽象、函数应用、let 表达式、递归函数定义、条件表达式以及二元操作。这是因为现代的 λ 演算教学不仅仅局限于纯粹的函数抽象和应用, 还需要涵盖更多的语言特性, 以便更好地模拟现代函数式编程语言的特性[[3]], 从而提高教学的实用性和针对性。

4.3 β -规约算法

因为现代函数式编程语言多为严格求值, 所以本项目采用了应用序列 (Applicative Order) 策略进行 β -规约[[1]]。该策略先对函数和参数进行求值, 然后再进行函数应用。

本项目的 β -规约算法实现了以下功能:

- 支持应用序列 (Applicative Order) 策略进行规约
- 实现 α -重命名避免变量捕获
- 实现替换 (Substitution) 算法
- 支持递归函数的规约
- 支持条件表达式的规约
- 支持二元操作的规约
- 支持 let 表达式的规约
- 支持 letrec 表达式的规约
- 支持类型检查, 确保表达式的类型正确性

核心算法为: 替换 (Substitution), 可以用如下的 λ 演算表达式来表示:

```
( $\lambda x. e1$ )  $e2 \rightarrow e1[x := e2]$ 
```

4.4 HM 类型推导 (Algorithm W)

4.4.1 简述

系统实现了 Hindley-Milner (HM) 类型系统的核心算法 —— Algorithm W[[4]] [[5]], 用于对 λ 表达式进行自动类型推导。HM 类型系统是现代函数式语言 (如 Haskell、OCaml、ML) 的理论基础[[3]]。本系统的 HM 类型推导模块主要用于: 验证用户输入的 λ 表达式是否类型正确, 并在可能的情况下推导出其最泛类型。

HM 类型系统有如下特点:

- 无需类型注解即可自动推导类型
- 支持多态 (Let-Polymorphism)
- 能推导出表达式的最泛类型 (Principal Type)

4.4.2 类型系统定义:

```

data Type
  = TInt
  | TBool

```

```

| TVar String
| TFun Type Type
deriving (Show, Eq)

```

系统支持以下类型构造:

- 基本类型: Int、Bool
- 类型变量: a、b、t1 等
- 函数类型: $\tau_1 \rightarrow \tau_2$

除以上静态类型, 系统也支持多态类型(Scheme):

```

data Scheme = Forall [String] Type

```

4.4.3 Substitutable 类型类

为了统一处理类型变量替换, 本系统实现了 Substitutable 类型类:

```

class Substitutable a where
  apply :: Subst -> a -> a
  ftv   :: a -> Set String

```

该类型类支持:

- 对 Type、Scheme、TypeEnv 进行替换
- 计算自由类型变量 (Free Type Variables)

这是 Algorithm W 的基础设施。

4.4.4 替换 (Substitution) 与合一 (Unification)

系统实现了 Robinson 合一算法[[3]], 用于求解类型约束:

```

unify :: Type -> Type -> Either String Subst

```

支持:

- 基本类型匹配
- 类型变量绑定 (bind)
- 函数类型的递归合一
- 无限类型检查 (Occurs Check)

替换组合使用:

```

composeSubst :: Subst -> Subst -> Subst

```

用于合成多个替换, 确保类型推导过程中替换的一致性。

4.4.5 实例化与泛化

4.4.5.1 实例化 (Instantiation)

将多态类型 $\forall a. a \rightarrow a$ 转换为具体类型:

```

instantiate :: Scheme -> Infer Type

```

4.4.6 泛化 (Generalization)

在 `let x = e1 in e2` 中, 将 `e1` 的类型泛化为 Scheme:

```

generalize :: TypeEnv -> Type -> Scheme

```

```

x = \y -> y

```

对于上述表达式, 得到的泛化类型为:

```

x :  $\forall a. a \rightarrow a$ 

```

4.4.7 模块价值

HM 类型推导模块为系统提供:

- 静态类型检查能力

- 自动类型推导能力
 - 多态支持 (Let-Polymorphism)
 - 递归函数类型推导 (LetRec)
 - 与 λ 演算规约引擎的深度结合
- 大幅度提升了系统的理论深度

5 数据结构设计

为了支持 λ 演算的解析、规约、类型推导与可视化, 本系统设计了一套结构清晰、语义严谨的数据结构体系。本节将介绍系统中最核心的四类数据结构: Type、Scheme、Subst、TypeEnv, 它们共同构成 HM 类型推导与 β -规约的基础。

5.1 Type: 静态类型

Type 是系统中表示类型的核心数据结构, 定义如下:

```
data Type
  = TInt
  | TBool
  | TVar String
  | TFun Type Type
  deriving (Show, Eq)
```

5.2 Scheme: 多态类型 (

Scheme 用于表示 HM 类型系统中的多态类型, 定义如下:

```
data Scheme = Forall [String] Type
```

5.3 Subst: 替换表

Subst 是类型推导中最关键的数据结构之一, 用于记录类型变量的替换关系, 定义如下:

```
type Subst = Map String Type
```

6 创新点

6.1 可视化 λ 演算规约

传统的 λ 演算教学通常依赖手工推导, 学生难以直观理解 β -规约的动态过程[[1]]。本项目通过可视化引擎, 将每一步规约过程直观显示, 并附有规约过程。这使得抽象的 λ 演算规约过程变得可观察、可追踪、可理解, 大幅提升教学效果。

6.2 支持部分现代函数式编程语言特性的 λ 演算

传统的 λ 演算教学通常局限于纯粹的函数抽象和应用, 而现代函数式编程语言 (如 Haskell[[2]]、OCaml) 引入了更多的语言特性, 如 let 表达式、递归函数定义、条件表达式以及二元操作等。本项目的 λ 演算教学平台支持这些现代函数式编程语言的特性[[3]], 使得教学内容更贴近实际编程语言的特性, 从而提高教学的实用性和针对性。

6.3 HM 类型系统的实现

虽然 HM 类型系统是不实现依赖类型的函数式编程语言的核心类型系统[[4]; [5]], 但在 λ 演算教学平台中, 常见的多为无类型 λ 演算, 而 HM 类型系统的引入使得教学平台能够支持类型检查和类型推导功能, 帮助学生理解类型系统的概念和作用[[3]], 并且为未来可能的依赖类型系统的引入奠定扎实了基础。

7 使用指南

7.1 快速开始

启动系统后, 用户将看到左侧实例栏和右侧主工作区。

7.1.1 输入表达式

在右侧工作区的输入框中输入 λ 演算表达式, 语法如下:

- 变量: x
- 函数抽象: $\lambda x \rightarrow x + 1$
- 函数应用: $f\ x$ 或 $f\ (g\ x)$
- 条件表达式: $\text{if cond then } e1 \text{ else } e2$
- let 表达式: $\text{let } x = e1 \text{ in } e2$
- 二元操作: $x + 1$ 、 $x > 0$ 等

7.1.2 执行操作

输入表达式后, 可选择以下三种操作之一:

- **单步规约**: 点击该按钮, 系统将调用 `/api/trace` 接口, 展示从输入表达式到最终值的完整规约过程, 每一步显示表达式、类型及使用的规约规则 (如 β -reduction)。
- **直接求值**: 点击该按钮, 系统将调用 `/api/eval` 接口, 直接显示表达式的最终值和类型。
- **类型推导**: 点击该按钮, 系统将调用 `/api/typecheck` 接口, 显示表达式的 Hindley-Milner 类型。

7.1.3 使用实例库

左侧栏提供常用 λ 演算实例, 点击即可将表达式填入输入框。用户也可添加自定义规则:

1. 点击“添加规则”按钮
2. 输入规则名称 (不能与保留符号冲突) 和对应的 λ 表达式
3. 保存后规则将出现在左侧栏, 并持久化保存在浏览器本地存储中

7.1.4 语法帮助

点击“语法”按钮可查看 λ 演算的基本语法说明。

8 安装与部署

8.1 环境要求

- 后端: GHC (Glasgow Haskell Compiler) ghc-9.6 或更高版本, Cabal 3.10+
- 前端: Node.js 16+, npm 8+

8.2 后端部署

```
# 克隆仓库
git clone https://github.com/X12101XX/lambdaLens.git
cd lambdaLens

# 安装依赖并构建
cabal build

# 运行后端服务器 (默认端口 3000)
cabal run lambdaLens
```

8.3 前端部署

```
cd frontend

# 安装依赖
npm install

# 开发模式运行 (默认端口 5173)
npm run dev

# 构建生产版本
npm run build
```

8.4 配置说明

前端默认连接 `http://localhost:3000` 作为后端 API 地址。如需修改, 可在前端环境变量或配置文件中设置。

9 示例演示

9.1 示例 1: 恒等函数

- 输入: $\lambda x \rightarrow x$
- 操作: 类型推导
- 结果: $a0 \rightarrow a0$ (类型变量 $a0$ 表示任意类型)

9.2 示例 2: 函数应用

- 输入: $(\lambda x \rightarrow x + 1) \ 2$ 操作: 单步规约 规约过程:
1. $(\lambda x \rightarrow x + 1) \ 2$ (类型: Int)
 2. $2 + 1$ (β -reduction)
 3. 3 (β -reduction, 最终结果)

9.3 示例 3: 条件表达式

输入: `\x -> if x > 0 then x else 0` 操作: 类型推导 结果: `Int`
→ `Int`

9.4 示例 4: 递归函数

输入: `letrec fact = \n -> if n < 1 then 1 else n * (fact (n - 1)) in fact 5` 操作: 直接求值 结果: `120` (类型: `Int`)

10 测试与验证

10.1 测试范围

系统已对以下功能进行测试:

- Parser 模块: 各类 λ 表达式的解析, 包括嵌套函数、括号优先级等
- Evaluator 模块: β -规约的正确性, α -重命名避免变量捕获, 递归函数规约
- API 模块: 三个接口的正常请求与异常处理
- 前端: 表达式输入、按钮交互、结果展示、自定义规则增删改查

11 局限性与未来工作

11.1 当前局限性

- 递归深度限制: 为防止无限递归, 系统对递归深度设有上限
- 不支持的特性: 目前不支持高阶类型、类型类、模式匹配等高级特性
- 数值范围: 整数运算受限于 Haskell `Int` 类型的范围
- 并发支持: 当前为单线程处理, 未考虑高并发场景

11.2 未来工作

- 支持更多语法特性: 如模式匹配、列表、元组等数据结构
- 优化可视化: 引入 AST 图形化展示、规约过程动画
- 增加教学功能: 添加练习题、自动评分、学习进度跟踪
- 多语言支持: 界面国际化, 支持中英文切换
- 部署优化: 提供 Docker 镜像, 简化部署流程

12 参考文献

- [1] Hendrik Pieter Barendregt, The Lambda Calculus: Its Syntax and Semantics. North-Holland.
- [2] Simon Peyton Jones, Haskell 98 Language and Libraries: The Revised Report. Cambridge University Press.
- [3] Benjamin C. Pierce, Types and Programming Languages. MIT Press.
- [4] J. Roger Hindley, “The Principal Type-Scheme of an Object in Combinatory Logic.”

- [5] Robin Milner, “A Theory of Type Polymorphism in Programming.”